



DESARROLLO BIG DATA CON PYTHON

TEMA 1. Plataforma y
Fundamentos de Python



Sobre el Formador

/ Iván Palomares Carrascosa

Ingeniero Superior (2009) y Doctor (2014) en Informática
Master en Sistemas Inteligentes (2011)

Ex-profesor de universidad e investigador (2012-2022)

- Machine Learning, Análisis y Ciencia de Datos
- Inteligencia Artificial Aplicada

Actualmente *freelance*:

- Formador a medida para empresas y organizaciones.
- Colaborador docente y consultor con escuelas de negocio.
- Escritor y redactor en tech blogs internacionales.
- Formador IT homologado por la EOI (Escuela de Organización Industrial)



Contenidos Tema 1

- / 1. Sintaxis de Python, Biblioteca estándar y módulos**
- / 2. Colecciones y Funciones**
- / 3. Clases y Objetos (POO)**
- / 4. Elementos de Programación Funcional en Python**
- / 5. Operaciones de Entrada-Salida y Gestión de Errores**
- / 6. Plataforma de Desarrollo VS Code: creación de scripts básicos locales y notebooks**

Enlace a Materiales:

[...]

REPOSITORIO DE DATASETS PARA EL CURSO: <https://github.com/gakudo-ai/open-datasets>

→ **Python** es actualmente el **lenguaje de programación más utilizado del mundo**. ¿El motivo? Su gran **versatilidad** y su **amplia comunidad online** con proyectos, documentación y recursos.

- ✓ Programación científica
- ✓ Desarrollo Web
- ✓ DevOps (desarrollo y operaciones)
- ✓ Administración de Sistemas
- ✓ **Data Science y Análisis → Big Data**
- ✓ **Machine Learning**
- ✓ **Inteligencia Artificial**

Curiosidades:

La curva de aprendizaje para desenvolverse programando en Python oscila entre los 2-6 meses



Es un **lenguaje de alto nivel**: diseñado para ser comprensible y legible por el ser humano, mediante la abstracción de conceptos y una sintaxis simplificada.

→ Independientemente del lenguaje de programación, todo programa se compone de un conjunto de **instrucciones**:

- ✓ **Asignación**: asignar a una variable un valor concreto o el de otra variable.
- ✓ **Entrada/salida**: transferencia de información entre dispositivos periféricos –*teclado, pantalla, etc.*– y la “memoria central” del programa.
- ✓ **Cálculo**: realización de operaciones aritméticas o más complejas entre variables y/o valores constantes.
- ✓ **De control**: modifican la secuencia de ejecución del programa, mediante **condiciones** y **bucles**.

```
nombre = "Iván"
edad = 38

ciudad = input("¿ En qué ciudad vives?")
print("Hola,", nombre, "de", ciudad, "!")

doble_edad = edad * 2
print("El doble de tu edad es:", doble_edad)

if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")

for i in range(3):
    print(f"Intento {i+1}: ¡Python es genial!")
```



1. Sintaxis Python, Biblioteca Estándar y Módulos

- Debido a su versatilidad y funcionalidad, además de ser de código abierto, existe un enorme número y variedad de bibliotecas o librerías que se pueden utilizar
- Una **biblioteca** es un conjunto de funcionalidades que permiten al usuario realizar tareas que sin ella no se podrían realizar, creando una interfaz de uso independiente.
 - Cada librería se compone de multitud de:
 - ✓ **Módulos:** forman parte de una librería, son archivos (.py) con código reusable, como clases, funciones o variables. “math” es un claro ejemplo de ello.
`import math`
 - ✓ **Funciones específicas:** se puede importar solamente la función que nos interese.
`from math import sqrt`
- Un **script** es un archivo (.py) hecho para ser ejecutado directamente en vez de ser un módulo importado.

¿Similitudes y diferencias entre un módulo y un script Python?



- Cada tipo de datos en Python tiene sus propias características y propiedades.
- El intérprete de Python normalmente discierne el tipo de dato de una variable, literal, resultado de una operación, etc.
 - De todos modos, a un mayor nivel de abstracción, todo se instancia como **objetos**.

TIPO ESCALAR	EXPLICACIÓN	EJEMPLO
int	Números enteros, sin límite de tamaño. Se pueden representar en decimal, binario (0b...), octal (0o...) y hexadecimal (0x...).	<pre>a = -1 diez_binario = 0b1010 diez_hex = 0xa</pre>
float	Números reales y con notación científica, representados en 32 o 64 bits.	<pre>real = 1.1 + 2.2 cientifico = 1.23E2</pre>
str	Cadenas de texto en formato Unicode, delimitadas por comillas simples o dobles.	<pre>hola = "Hola Python" hola2 = 'Hi, \'Python\''</pre>
bool	Solo tiene dos valores: True o False. Algunos objetos vacíos o cero son considerados False.	<pre>print(1 > 0) print(bool([]))</pre>

Métodos útiles para *strings*

Método	Descripción
upper()	Convierte una cadena a mayúsculas, devolviendo una copia en mayúsculas.
lower()	Convierte una cadena a minúsculas, devolviendo una copia en minúsculas.
capitalize()	Convierte la primera letra de una cadena a mayúscula.
swapcase()	Devuelve una copia de la cadena con mayúsculas cambiadas por minúsculas y viceversa.
title()	Convierte la primera letra de cada palabra en una cadena a mayúscula.
isupper()	Devuelve True si todos los caracteres están en mayúsculas, False en caso contrario.
islower()	Devuelve True si todos los caracteres están en minúsculas, False en caso contrario.
strip()	Elimina los espacios en blanco al inicio y al final de la cadena.
rstrip()	Devuelve una nueva cadena con los espacios en blanco del final eliminados.
lstrip()	Devuelve una nueva cadena sin los espacios en blanco del principio.
replace(sub, nuevo)	Devuelve una copia de la cadena con todas las ocurrencias de sub reemplazadas por nuevo.
replace(sub, nuevo, n)	Reemplaza hasta n ocurrencias de sub por nuevo.
split(sep=None, maxsplit=-1)	Devuelve una lista de palabras usando sep como separador. Si sep no se especifica, se eliminan espacios en blanco. Si se usa sin argumentos, elimina espacios en blanco y caracteres especiales como \n \t etc.
9 maxsplit	Permite indicar el número máximo de divisiones a realizar en split().

→ Conversión implícita

- Ocurren de forma automática, cuando se realiza una operación entre diferentes tipos y son convertidos al tipo de mayor rango (riqueza de valores)

TIPO 1	TIPO 2	RESULTADO
booleano	int	int
booleano	float	float
int	float	float

```
valor = 10 + True  
valor
```

11

```
otro_valor = valor + 0.1  
otro_valor
```

11.1

1. Sintaxis Python, Biblioteca Estándar y Módulos

Una excepción no permitida en Python es el casting **string** → **float**, pudiendo dar un **ValueError** para cadenas alfanuméricas

→ Conversión explícita (*casting*)

- A menudo recomendadas para tener un mayor control sobre cómo cambian los tipos de datos en diferentes variables del código.
- Se hace de forma deliberada mediante funciones o expresiones que transforman un tipo de dato en otro, gracias al tipado dinámico del lenguaje Python

CASTING	MÉTODO	EJEMPLO DE CÓDIGO			Python
entero (int)	<code>int(valor)</code>	<code>x = int(3)</code>	<code>y = int(3.5)</code>	<code>z = int("3")</code>	
. flotante (float)	<code>float(valor)</code>	<code>x = float(3)</code>	<code>w = float("3.5")</code>	<code>z = float("3")</code>	<code>x = "47.5"</code>
cadena (str)	<code>str(valor)</code>	<code>x = str("d4")</code>	<code>y = str(3)</code>	<code>z = str(5.0)</code>	
lista (list)	<code>list(secuencia)</code>		<code>tup = (1, 2, 3)</code>	<code>lst = list(tup)</code>	
tupla (tuple)	<code>tuple(secuencia)</code>		<code>lst = [4, 5, 6]</code>	<code>tup = tuple(lst)</code>	
conjunto (set)	<code>set(secuencia)</code>	<code>a = "Básico Python"</code>	<code>b = set(a)</code>	<code>print(type(b), b)</code>	
diccionario (dict)	<code>dict(lista_de_pares_clave_valor)</code>	<code>lst = [("nombre", "Diego"), ("edad", 46)]</code>	<code>dic = dict(lst)</code>		
booleano (bool)	<code>bool(valor)</code>	<code>val = 0</code>	<code>bool_val = bool(val)</code>		

→ Particularidades de los bucles en Python

- Python destaca por una sintaxis que permite definir bucles de forma clara y a veces muy concisa.

- Se pueden recorrer rangos de números sencillos con **range()**

```
for i in range(1, 6):  
    print(i)
```

```
# Imprimirá:  
# 1, 2, 3, 4, 5
```

```
frutas = ["manzana", "banana", "cereza"]  
for i in range(len(frutas)):  
    print(i, frutas[i])
```

- Se puede usar **enumerate()** con colecciones iterables, principalmente *list* y *tuple*, pero también diccionarios

```
frutas = ["manzana", "banana", "cereza"]  
for i, fruta in enumerate(frutas):  
    print(i, fruta)
```

→ Particularidades de los bucles en Python

- Python destaca por una sintaxis que permite definir bucles de forma clara y a veces muy concisa.
- **zip()** permite, como si fuera los dos lados de una cremallera, recorrer dos colecciones – normalmente listas, tuplas y cadenas- al unísono:

```
nombres = ["Ana", "Juan", "Luis"]
edades = [25, 30, 22]

for nombre, edad in zip(nombres, edades):
    print(nombre, "tiene", edad, "años")
```

- Se puede iterar en orden inverso con **reversed()**
- Se pueden recorrer los pares de un diccionario con **items()**
- Comprensión de listas: es un bucle compacto de una línea que crea una lista en una sola línea

```
cuadrados = [x**2 for x in range(5)]
print(cuadrados)
```

1. Sintaxis Python, Biblioteca Estándar y Módulos



Librería de código abierto muy orientada al ámbito de Data Science y Machine Learning, ofreciendo estructuras poderosas y flexibles que facilitan la manipulación y tratamiento de los datos. Esta librería surgió por la creciente necesidad de aunar en una única librería todo lo necesario para disponer en una misma herramienta todas las funcionalidades que se necesitan para analizar datos (carga, modelado, análisis, manipulación y preparación).

Pandas



Biblioteca muy utilizada para realizar operaciones matemáticas y científicas en matrices y vectores, ya que proporciona una interfaz flexible para trabajar con datos numéricos de una manera rápida y eficiente gracias a los arreglos multidimensionales dado que permite crear y operar con matrices y vectores sorprendentemente útiles para cálculos numéricos y científicos, además del manejo de funciones que proporcionan una amplia gama de funciones.

NumPy



Biblioteca que se utiliza para realizar cálculos científicos y técnicos gracias a una amplia gama de funciones y herramientas que facilitan el procesamiento de datos, su optimización, interpolación, álgebra lineal, estadística y muchas otras operaciones matemáticas. Está basada en la biblioteca NumPy, aunque complementa sus capacidades, brindando funcionalidad adicional y avanzada a la hora de realizar análisis numéricos y computacionales complejos de manera eficiente y rápida.

SciPy

1. Sintaxis Python, Biblioteca Estándar y Módulos



Biblioteca de aprendizaje automático de código abierto que proporciona gran número de algoritmos de aprendizaje supervisado y no supervisado, así como herramientas para preprocesamiento de datos, evaluación de modelos y selección de características, utilizado por una gran variedad de aplicaciones de aprendizaje automático, incluyendo clasificación, regresión, agrupamiento, reducción de dimensionalidad y detección de anomalías.

Scikit-Learn



Biblioteca utilizada para la creación y entrenamiento de modelos de aprendizaje automático y redes neuronales, permitiendo desarrollar aplicaciones de **IA (Inteligencia Artificial)**, así como para realizar cálculos numéricos de alto rendimiento y utilizar técnicas como el aprendizaje profundo y el procesamiento de imágenes en la industria que se dedica a resolver problemas complejos, como reconocimiento de voz, visualización compleja de datos y otros tantos.

TensorFlow



Biblioteca gratuita utilizada para el desarrollo de redes neuronales de alto nivel escrita en Python ya que proporciona una interfaz fácil de usar para construir y entrenar modelos de redes neuronales, facilitando la implementación de redes neuronales profundas y simplificando el proceso de entrenamiento y evaluación de los modelos utilizando diferentes tipos de **capas** como capas densas, capas convolucionales y capas recurrentes.

Keras

1. Sintaxis Python, Biblioteca Estándar y Módulos

matplotlib Biblioteca de visualización de datos que se utiliza para crear gráficos en 2D y 3D con la cual podemos crear una amplia variedad de gráficos, incluyendo gráficos de líneas, de dispersión, de barras, de superficie y muchos más. Además, se pueden añadir leyendas, títulos y multitud de elementos. La biblioteca está construida sobre Numpy y trabaja de forma natural con los vectores y matrices (arrays).



Matplotlib



Biblioteca dedicada a la visualización de datos, para lo que dispone de herramientas de creación de gráficos interactivos de diversos tipos como los gráficos de líneas, de barras, de dispersión, de área y mucho más.

Esta biblioteca es muy popular gracias a su facilidad de uso y la gran variedad de gráficos.

Plotly



Biblioteca de código abierto de aprendizaje automático y procesamiento de datos que se pueden utilizar para la implementación de aplicaciones de inteligencia artificial, como el desarrollo de modelos de aprendizaje profundo, procesamiento de imágenes y el procesamiento de lenguaje natural, ya que permite definir y entrenar redes neuronales de manera rápida y eficiente, aprovechando la capacidad de cómputo de las CPUs.

Pytorch

Principales colecciones de objetos en Python

COLECCIÓN	EXPLICACIÓN	EJEMPLO
list	Colección ordenada y mutable de elementos. Puede contener diferentes tipos de datos.	<code>numeros = [1, 2, 3, 4]</code> <code>mixta = [1, "Hola", 3.5, True]</code>
tuple	Similar a una lista, ordenada pero inmutable (no se puede modificar después de su creación).	<code>coordenadas = (10, 20)</code> <code>dias = ("Lunes", "Martes", "Miércoles")</code>
set	Conjunto desordenado de elementos únicos. No permite duplicados.	<code>vocales = {'a', 'e', 'i', 'o', 'u'}</code> <code>numeros_unicos = {1, 2, 2, 3, 4}</code>
dict	Colección de pares clave-valor , donde cada clave es única y se usa para acceder a su valor asociado.	<code>persona = {"nombre": "Ana", "edad": 25}</code> <code>puntos = {"x": 10, "y": 20}</code>

- Las **funciones** son el principal mecanismo para organizar, y sobretodo **reutilizar**, código en Python.
- Si crees que vayas a necesitar repetir un mismo fragmento de código múltiples veces, considera definir una función en torno al mismo.
- Las funciones se declaran con la palabra reservada **def**, y normalmente se sale de ellas con la palabra reservada **return**

```
def mi_funcion(x, y, z=1.5):  
    if z > 1:  
        return z * (x + y)  
    else:  
        return z / (x + y)
```

- Los **argumentos posicionales** son obligatorios. Hay que pasarlos en la llamada a la función.
- Los **argumentos de palabra clave** (keyword arguments) van tras los posicionales, son opcionales y llevan un valor por defecto, en caso de no especificarse en la llamada a la función.

```
def mi_funcion(x, y, z=1.5):  
    if z > 1:  
        return z * (x + y)  
    else:  
        return z / (x + y)
```

x, y son argumentos
posicionales

z es un argumento de
palabra clave (keyword
argument)

→ Más características de las funciones en Python

- Permiten devolver varios resultados simultáneamente.
- No perdamos de vista que las funciones en Python son **objetos**, por lo que permiten un uso más flexible que otros lenguajes, por ejemplo creando listas de funciones para construir flujos de procesamiento o *pipelines*.
- También se puede usar una función como argumento en la llamada a otra función. **La función se pasa como objeto, siendo una referencia, no una llamada inmediata a la función.**

```
def f():  
    a = 5  
    b = 6  
    c = 7  
    return a, b, c
```

```
flujo_limpieza = [str.strip, quitar_especiales, str.title]
```

```
for x in map(quitar_especiales, estados):  
    print(x)
```

Programación Orientada a Objetos (POO): permite trasladar al código mecanismos conceptuales similares a los de entidades del mundo real. Objetos con sus propias funciones y propiedades.

Beneficios:

Encapsulamiento	Se empaqueta el código dentro de una unidad (clase).
Abstracción	Permite modelar objetos/conceptos como clases y simplificar el programa.
Herencia	Reutilización de código al crear una clase que hereda los atributos y comportamientos de otra.
Polimorfismo	Creación de múltiples objetos diferentes a partir de un mismo código.

Desventajas:

Sobrecarga de rendimiento	Esto se acentúa en el caso de Python , al ser un lenguaje interpretado y no compilado. Las instancias de objetos tienen su propio diccionario (<code>__dict__</code>) para guardar atributos, y supone un despilfarro de memoria.
Complejidad	Para procesos sencillos, como operaciones en una calculadora, es posible que sea más conciso y directo codificarlos sin necesidad de objetos.
Curva de aprendizaje	Conceptos difíciles de dominar, como la herencia múltiple, metaclasses, etc.
Herencia múltiple	Puede dar lugar a situaciones como el “problema del diamante”

- El punto “.” se usa para acceder a los miembros -atributos y métodos- de un objeto. Esto ya lo hemos hecho, sobretodo con métodos, muchas veces incluso con los tipos de datos más básicos.

```
nombre.upper()
```

- Vamos a definir una primerísima clase muy simple, que representa a un gato:

Declaración de una clase

```
class Gato:
    def maullar(self):
        print("meow")
```

Creación y uso de un objeto de la clase "gato"

```
doraemon = Gato()
doraemon.maullar()
```

Es una clase sin atributos, y con un solo método, maullar.

Lo que distingue a un método de una clase de otras funciones, es el parámetro "self", primer y obligatorio parámetro en todos los métodos de una clase.

3. Clases y Objetos (POO)

- ¿Cómo dotar a objetos de nuestra clase con **atributos**? Mediante un método especial, el **constructor**, que es llamado automáticamente cuando creamos un nuevo objeto de la clase.
- Al crear el objeto, es el momento de asignarle, si lo deseamos, atributos con sus valores asociados.

```
class GatoParlante:
```

```
    def __init__(self, nombre):
        self.nombre = nombre
```

```
    def maullar(self):
        print(f"{self.nombre}, ¡así me llamo!")
```

```
meowth = GatoParlante("Micho")
meowth.maullar()
```

El constructor es un método especial llamado al inicializar un objeto, llamado

`__init__`

Asignación de valor al atributo

Ahora hay que instanciar nuevos objetos de esta clase con los argumentos requeridos para formar sus atributos.

Acceso al método. Si no se añaden los (), no se devuelve nada

Acceso a sus atributos. En Python también se les suele llamar "campos de datos" (data fields)

```
class Gato:
```

```
    def __init__(self, nombre, raza):
        self.nombre = nombre
        self.raza = raza
```

```
    def maullar(self):
        print("Meow meow")
```

```
gato1 = Gato("Luna", "Bengalí")
gato1.maullar()
```

```
print(gato1.nombre)
print(gato1.raza)
```

Objeto de una clase Gato "ampliada" con más atributos

3. Clases y Objetos (POO)

- Los nombres de las variables no tienen por qué coincidir en la inicialización de atributos

```
class Dueño:
    def __init__(self, nombre, direccion, numero_tlf):
        self.nombre = nombre
        self.direccion = direccion
        self.tlf = numero_tlf
```

El atributo inicializado en el objeto no se llamará "numero_tlf" sino "tlf"

- Pueden existir **relaciones entre objetos de diferentes clases**

```
class Gato:
    def __init__(self, nombre, raza, dueño):
        self.nombre = nombre
        self.raza = raza
        self.dueño = dueño
```

```
dueño1 = Dueño("Ivan", "Kioto", 123456789)
gato1 = Gato("Luna", "Bengalí", dueño1)
```

```
dueño2 = Dueño("María", "Granada", 909090909)
gato2 = Gato("Micho", "Persa", dueño2)
```

- En la **agregación**, un objeto de una clase forma parte de otro de otra clase, pero el objeto "contenido" puede existir independientemente

```
class Gato:
    def __init__(self, nombre, raza, dueño):
        self.nombre = nombre
        self.raza = raza
        self.dueño = dueño
```

```
dueño1 = Dueño("Ivan", "Kioto", 123456789)
gato1 = Gato("Luna", "Bengalí", dueño1)

dueño2 = Dueño("María", "Granada", 909090909)
gato2 = Gato("Micho", "Persa", dueño2)
```

En la **composición**, un objeto que forma parte de otro no puede existir por sí solo, y sería destruido si el objeto que lo contiene se elimina → es una **dependencia más fuerte**

```
class Salario:
    def __init__(self,pago):
        self.pago=pago

    def get_total(self):
        return (self.pago*12)

class Empleado:
    def __init__(self,pago,bonus):
        self.pago=pago
        self.bonus=bonus
        self.obj_salario=Salario(self.pago)
```

```
def salario_anual(self):
    return "Total: " +
    str(self.obj_salario.get_total()+self.bonus)

obj_emp=Empleado(100,10)
print (obj_emp.salario_anual())
```

3. Clases y Objetos (POO)

- Relaciones de **uno a muchos**. En Python consisten en que un objeto de una clase tenga como uno de sus atributos una *colección* de objetos de otra clase.

```
class Biblioteca:
    def __init__(self, nombre, ubicacion):
        self.nombre = nombre
        self.ubicacion = ubicacion
        self.libros = []

    def agregar_libro(self, libro):
        self.libros.append(libro)

    def mostrar_libros(self):
        return f"Libros en {self.nombre}:\n" +
            "\n".join(str(libro) for libro in self.libros)

    def __repr__(self):
        return f"{self.nombre} está ubicada en {self.ubicacion}"
```

Aquí damos forma a la relación uno a muchos entre Biblioteca y otra clase, Libro, que crearemos a continuación

join() es muy útil para adjuntar en una cadena a imprimir una colección de elementos

Un método especial para definir la representación de un objeto de una clase como un string

```
class Libro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor

    def __repr__(self):
        return f"'{self.titulo}' por {self.autor}"
```

A este método, al igual que __init__, se le llama de forma implícita, si hacemos str(libro), asumiendo que la variable 'libro' contiene un objeto Libro

3. Clases y Objetos (POO)

```
# Crear una biblioteca
mi_biblioteca = Biblioteca("Biblioteca Central", "Calle Desengaño 21")

# Crear algunos libros
libro1 = Libro("Cien años de soledad", "Gabriel García Márquez")
libro2 = Libro("Don Quijote de la Mancha", "Miguel de Cervantes")

# Agregar los libros a la biblioteca
mi_biblioteca.agregar_libro(libro1)
mi_biblioteca.agregar_libro(libro2)

# Mostrar información de la biblioteca y sus libros
print(mi_biblioteca)
print(mi_biblioteca.mostrar_libros())
```

- La **herencia** permite a una **clase “Hija”** o **subclase** heredar atributos y métodos de otra clase, denominada **clase “Padre”** o **superclase**:

```
class MaterialBiblioteca:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor

    def __repr__(self):
        return f"{self.titulo} por {self.autor}"

    def ver_titulo(self):
        return f"Título: {self.titulo}"
```

Hereda de

```
class Libro(MaterialBiblioteca):
    def __init__(self, titulo, autor, genero):
        super().__init__(titulo, autor) # Constructor superclase
        self.genero = genero # Atributo propio de la subclase

    def info_completa(self):
        return f"{super().__str__()} | Género: {self.genero}"
```

→ Funciones anónimas (lambda)

- Son funciones sin nombre, escritas de modo compacto en una sola instrucción, cuyo resultado es el valor que devuelven tras aplicar una expresión.
- Se declaran con la palabra reservada **lambda**.
- Son muy comúnmente usadas con **funciones de orden superior** como *map()*, por ejemplo para aplicar una misma transformación a toda una colección.
- La expresión puede acomodar un operador ternario, por ejemplo.

lambda argumentos: expresión

```
lambda x: x * 2  
lambda x, y: x + y  
lambda x: x if x > 0 else 0
```

→ Funciones de orden superior

- Son funciones que “trabajan usando otras funciones”: reciben otra función como argumento.
- Se podría decir que las funciones de orden superior “tratan a las funciones que usan como si fueran datos”.

Función	¿Qué hace?
<code>map(func, iterable)</code>	Aplica func a cada elemento del iterable
<code>filter(func, iterable)</code>	Filtra los elementos según lo que devuelva func
<code>sorted(iterable, key=func)</code>	Ordena usando func como criterio
<code>reduce(func, iterable)</code>	Acumula los elementos usando func

Ejemplos habituales de programación funcional en Python

1. Transformar datos con *map()*

`map(func, iterable)`

```
numeros = [1, 2, 3, 4, 5]

# Multiplicamos cada número por 2 usando una función lambda
dobles = list(map(lambda x: x * 2, numeros))

print(dobles)
```

2. Filtrar únicamente edades de personas adultas con *filter()*:

```
edades = [15, 22, 17, 30, 12, 45]

# Conservaremos únicamente los números que sean mayores o iguales a 18
mayores_de_edad = list(filter(lambda x: x >= 18, edades))

print(mayores_de_edad)
```

Ejemplos habituales de programación funcional en Python

3. Suma acumulativa con *reduce()*

```
from functools import reduce

valores = [10, 20, 30, 40]

# Sumamos el primero con el segundo, y ese total con el siguiente (operación asociativa)
suma_total = reduce(lambda acumulador, actual: acumulador + actual, valores)

print(suma_total)
```

4. Ordenar palabras por longitud con *sorted()*:

```
palabras = ["elefante", "sol", "manzana", "pez"]

# Ordenamos la lista basándonos exclusivamente en la longitud (len) de cada palabra
orden_por_longitud = sorted(palabras, key=lambda p: len(p))

print(orden_por_longitud)
```

Entrada/Salida eficiente → El problema de la memoria

- **Context Managers:** la estructura `with open()` se usa para garantizar el cierre automático de archivos al terminar de leer/escribir o si ocurre un error, liberando recursos críticos.

- **Lectura Perezosa (Lazy):** Iterar línea por línea: `for linea in archivo`. Evita cargar por ejemplo un dataset masivo de 50GB en la memoria RAM simultáneamente.

- **Estructuras Óptimas:** Usar colecciones como `set()` (conjuntos) asegura unicidad a velocidades ultrarrápidas ($O(1)$) al procesar logs.

```
# Extraer usuarios únicos de un log masivo
usuarios_unicos = set()

# LECTURA: Procesamiento línea por línea
with open("terabytes_de_logs.csv", "r") as f_in:
    for linea in f_in:
        usuario = linea.split(",")[0]
        usuarios_unicos.add(usuario)

# ESCRITURA: Guardar el resultado
with open("reporte_final.txt", "w") as f_out:
    total = len(usuarios_unicos)
    f_out.write(f"Total usuarios: {total}")
```

Protegiéndose ante datos "sucios"

- **Datos impredecibles:** En ingestas masivas (APIs, *web scraping*, sensores), raramente no habrá valores nulos, columnas vacías o tipos de datos incorrectos.
- **Tolerancia a fallos:** El bloque `try / except` captura excepciones predecibles. Evita que un pipeline que lleva 4 horas ejecutándose colapse por un solo registro anómalo.
- **Aislamiento:** Se registra (loguea) el dato problemático para una auditoría posterior, pero el ciclo `for` continúa con el resto de los registros sanos.

```
datos_crudos = ["100", "250", "N/A", "400", ""]
datos_limpios = []

for valor in datos_crudos:
    try:
        # Intentamos convertir el texto a número
        numero = int(valor)
        datos_limpios.append(numero)
    except ValueError:
        # Si falla (ej: "N/A"), NO se detiene la ejecución
        print(f"[ALERTA] Dato ignorado: '{valor}'")

print(f"Éxito: Procesados {len(datos_limpios)} reg.")
```

6. Plataforma de Desarrollo: VS Code

Los entornos más utilizados en la actualidad pueden agruparse en dos tipos

- **Entornos de Desarrollo Integrados (IDEs):** diseñados para el desarrollo de aplicaciones profesionales, con herramientas avanzadas de depuración, refactorización y gestión de proyectos complejos
 - ✓ **PyCharm:** desarrollado por JetBrains
 - ✓ **VS Code:** ligero y muy utilizado
 - ✓ **IDLE:** IDE por defecto que ofrece Python
 - ✓ **Otros:** Spyder, Thonny
- **Entornos interactivos y en la nube:** enfocados en la experimentación, análisis de datos, ciencia de datos y machine learning, permite ejecución paso a paso.
 - ✓ **Jupyter Notebooks:** permite ejecutar código en celdas e intercalar texto. Versiones en la nube incluyen **Google Colab** y **Anaconda JupyterLab**



Laboratorio: Creación de scripts básicos locales en VS Code y ejecución de los primeros notebooks en Google Colab.

